

Full Stack Development with MERN

Functional Specification Documentation (FSD)

Date:	30 September 2026
Team ID:	SWTID-2026-1128
Project Title:	Auto Ticket Classification using Flow Designer
Project Stack:	MERN (MongoDB, Express.js, React, Node.js) & Flow Automation Engine

1. Introduction

Project Title: Auto Ticket Classification using Flow Designer

Team Members:

- Aravind S (Leader)

Roles:

- **Full Stack Developer:** Architected MERN application, configured Flow Designer trigger engine, and set up REST API endpoints.
- **Frontend Engineer:** Designed responsive React.js web portal, ticket status dashboards, and Chart.js analytics visuals.
- **Backend & Database Developer:** Implemented Node.js / Express API services, MongoDB schemas, and JWT token authentication middleware.
- **QA & Automation Engineer:** Executed UAT test suites, NLU model performance validation, and API load testing.

2. Project Overview

Purpose: The school IT helpdesk receives multiple support requests daily from students and teachers related to common issues such as network connectivity, hardware failures, account access problems, and system performance issues. Currently, IT staff manually review each ticket to identify the issue type, select the correct category and subcategory, and inform the caller about ticket creation. This manual process is time-consuming, error-prone, and not scalable as request volumes increase. The purpose of this project is to implement an automated classification and routing system leveraging Flow Designer and MERN stack services to eliminate triage delays, streamline queue management, and dispatch instant notifications to stakeholders.

Key Features & Functionalities:

- **Automated Ticket Ingestion:** Single-page React portal allowing students and teachers to submit issues with diagnostic attachments.
- **NLU & Flow Classification:** Automated text parsing engine predicting Category, Subcategory, and Priority with over 94% accuracy.
- **Smart Queue Assignment:** Dynamic routing matrix directing classified tickets straight to specialized IT support queues (Network, Hardware, Accounts, Performance).
- **Instant Caller Notifications:** Automated email/SMS status dispatches providing ticket tracking IDs and estimated resolution SLAs upon submission.
- **Role-Based Dashboards:** Interactive analytics views for administrators, technicians, and callers to monitor ticket throughput and SLA compliance.

3. Architecture

The application follows a standard 3-tier MERN stack architecture integrated with an automated Flow Designer classification engine:

- **Frontend Architecture (React.js):** Built using React (v18) functional components and React Hooks (useState, useEffect, useContext) for real-time state management. React Router (v6) handles client-side routing between portal submission forms, caller tracking views, and administrative dashboards. Axios is utilized for asynchronous HTTP REST calls with interceptors for JWT token attachment.
- **Backend Architecture (Node.js & Express.js):** Designed as a modular, RESTful API service using Node.js and Express.js. Controller-Service-Repository pattern is followed to separate route handling, flow automation logic, and database operations. Express middleware manages request logging, CORS, input sanitization, error handling, and JWT token authentication.
- **Database Architecture (MongoDB):** MongoDB (v6.0) multi-tenant document database managed via Mongoose ODM. Schema validation enforces strict data types, required fields, and index constraints on ticket IDs and user emails. Collections include Users, Tickets, Categories, and FlowLogs.

4. Setup Instructions

Prerequisites:

- Node.js (v18.x or higher) and npm (v9.x or higher)
- MongoDB Server (v6.0 local instance or MongoDB Atlas URI)
- Git Version Control
- Postman or cURL for API testing

Step-by-Step Installation Guide:

1. Clone Repository: `git clone https://github.com/school-it/auto-ticket-classification-mern.git`
2. Navigate to Project Root: `cd auto-ticket-classification-mern`
3. Setup Backend Environment: `cd server && npm install`
4. Configure Server .env File: Create a .env file in server/ with `PORT=5000`, `MONGO_URI=mongodb://localhost:27017/it_helpdesk`, `JWT_SECRET=supersecretkey`, and `SMTP_HOST=smtp.mailtrap.io`
5. Setup Frontend Environment: `cd ../client && npm install`
6. Configure Client .env File: Create a .env file in client/ with `REACT_APP_API_BASE_URL=http://localhost:5000/api`

5. Folder Structure

The repository is divided into *client/* (React frontend) and *server/* (Node.js backend) directories:

```
auto-ticket-classification-mern/
├── client/ # React Frontend Application
│   ├── public/ # Static assets and index.html
│   └── src/
│       ├── components/ # Reusable UI components (Navbar, Sidebar, TicketCard)
│       ├── pages/ # Page views (Home, SubmitTicket, Dashboard,
│       │       TicketDetails)
│       ├── context/ # React Context API (AuthContext, TicketContext)
│       ├── services/ # Axios API service calls
│       ├── utils/ # Helper utilities & date formatters
│       ├── App.js # Route definitions & layout wrappers
│       ├── index.js # React application entry point
│       └── package.json
└── server/ # Node.js & Express Backend Application
    ├── config/ # Database connection & Flow Designer rules
    ├── controllers/ # Request handlers (authController, ticketController)
    ├── middleware/ # JWT verification, error handlers, input validator
    ├── models/ # Mongoose schemas (User.js, Ticket.js, FlowLog.js)
    ├── routes/ # API endpoints (authRoutes.js, ticketRoutes.js)
    ├── services/ # Classification engine & Flow Designer trigger logic
    ├── server.js # Express app initialization & server startup
    └── package.json
```

6. Running the Application

Follow these commands to start the local development environment:

- **Backend Server Execution:**

```
cd server
npm start # Starts Express server on http://localhost:5000
```

- **Frontend Server Execution:**

```
cd client
npm start # Starts React dev server on http://localhost:3000
```

7. API Documentation

Below is the RESTful API documentation for endpoints exposed by the Node.js/Express backend:

Method	Endpoint	Description	Auth Required	Example Response
POST	/api/auth/login	Authenticate user and issue JWT bearer token	No	{ "token": "eyJhbGciOi...", "user": { "id": "123", "role": "student" } }
POST	/api/tickets	Submit new ticket & trigger Flow auto-classification	Yes (JWT)	{ "ticketId": "INC00123", "category": "Network", "subcategory": "Wireless", "priority": "High" }
GET	/api/tickets	Fetch all submitted tickets with filters & pagination	Yes (JWT)	[{ "ticketId": "INC00123", "title":

8. Authentication

Authentication and authorization are managed using JSON Web Tokens (JWT) and bcrypt password hashing:

- **Token Management:** Upon successful login via `/api/auth/login`, the server issues a signed JWT token containing user ID, email, and role claims (Student, Teacher, Technician, Admin).
- **Authorization Middleware:** Protected API endpoints pass incoming requests through an `authenticateToken` Express middleware that verifies token validity via HTTP Bearer header.
- **Role-Based Access Control (RBAC):** Admin and technician actions (such as manual category overrides or system configuration changes) are restricted via `authorizeRole(['admin', 'technician'])` middleware wrappers.

9. User Interface

The user interface features three primary modules designed using Bootstrap 4 and React components:

- **Student/Teacher Ticket Portal:** Clean submission form with real-time text input sanitization, category recommendation preview, and file upload zone.
- **Technician Triage Dashboard:** Interactive queue list displaying auto-classified tickets, NLU confidence scores, priority tags, and single-click manual override controls.
- **Analytics & Executive Summary View:** Real-time charts illustrating ticket resolution throughput, classification accuracy trends, and SLA breach monitors.

10. Testing

Testing strategy encompassed automated unit tests, integration tests, and UAT scenarios:

- **Unit Testing:** Jest & React Testing Library were used to test component rendering and form validation logic.
- **API Integration Testing:** Supertest & Postman test suites validated Express API routes, controller responses, and JWT middleware behavior.
- **Flow & NLU Testing:** Benchmark test set of 500 ticket descriptions evaluated classification accuracy (achieving 94.6% validation accuracy) and sub-2 second response latency.

11. Screenshots or Demo

Application Demonstration & Access Details:

- **Live Application Demo:**
https://drive.google.com/file/d/1PWLpYeR-Xnhan0loHXaOgXhxx7przKs9/view?usp=drive_link
- **Source Code Repository:** <https://github.com/aravinth-offix/Auto-Ticket>
- **Screenshots:**
https://drive.google.com/drive/folders/1cSm0jBx09XdBGwjBzkeA0C1K8OF7vB62?usp=drive_link

12. Known Issues

Current known limitations and edge cases under active optimization:

- **Ambiguous Short Descriptions:** Tickets with extremely brief descriptions (e.g., 'It is broken') yield NLU confidence scores below 0.70 and fall back to the manual review queue.
- **Combined Fault Reports:** When a ticket describes two simultaneous issues (e.g., 'Projector off and Wi-Fi disconnected'), the model prioritizes the primary keyword root cause.

13. Future Enhancements

Planned features for future release iterations:

- **Virtual Agent / Chatbot Integration:** Deploy an interactive React chatbot component providing instant automated troubleshooting steps prior to ticket creation.
- **Multilingual NLU Support:** Incorporate multi-language model processing to support non-English ticket submissions from international students.
- **Predictive Knowledge Base Suggestions:** Automatically surface relevant Knowledge Base articles to submitters based on extracted issue context.